2022 — 2023

Graph-native route & cost *optimization platform*

Client A global logistics enterprise

Source Replacing a relational ERP route engine

Neo4j Enterprise

Google Cloud · GKE · Cloud Run

Kafka · Striim CDC

Python · NestJS · Next.js

Modeling a multi-modal shipping network as a graph — so end-to-end routes, schedules and costs can be served in milliseconds.

A global logistics enterprise needed to retire a monolithic relational route engine in favour of a graph-native platform that could compose ocean, feeder and inland legs into millions of weekly end-to-end routes, attach multi-tier cost, and react to upstream ERP changes in near-real-time.

I am data engineer supporting the following area of in-bound message flow, change-data-capture topology, Neo4j cluster design on GCP, and the orchestration model behind both initial-load and dynamic-update pipelines.

2.9 M
END-TO-END ROUTES GENERATED PER WEEK

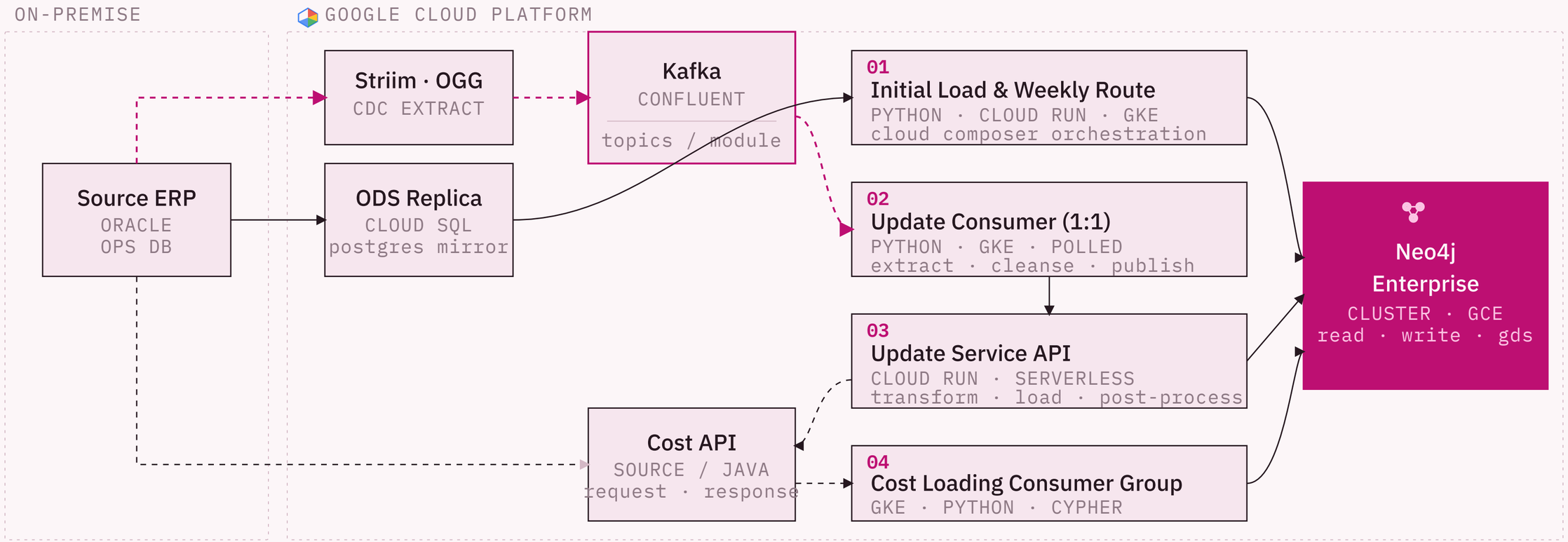
9
GRAPH FUNCTIONAL MODULES

~10 min
FOUNDATION DATA LOAD (6 MONTHS)

3 +1
CORE NEO4J NODES + GDS REPLICA

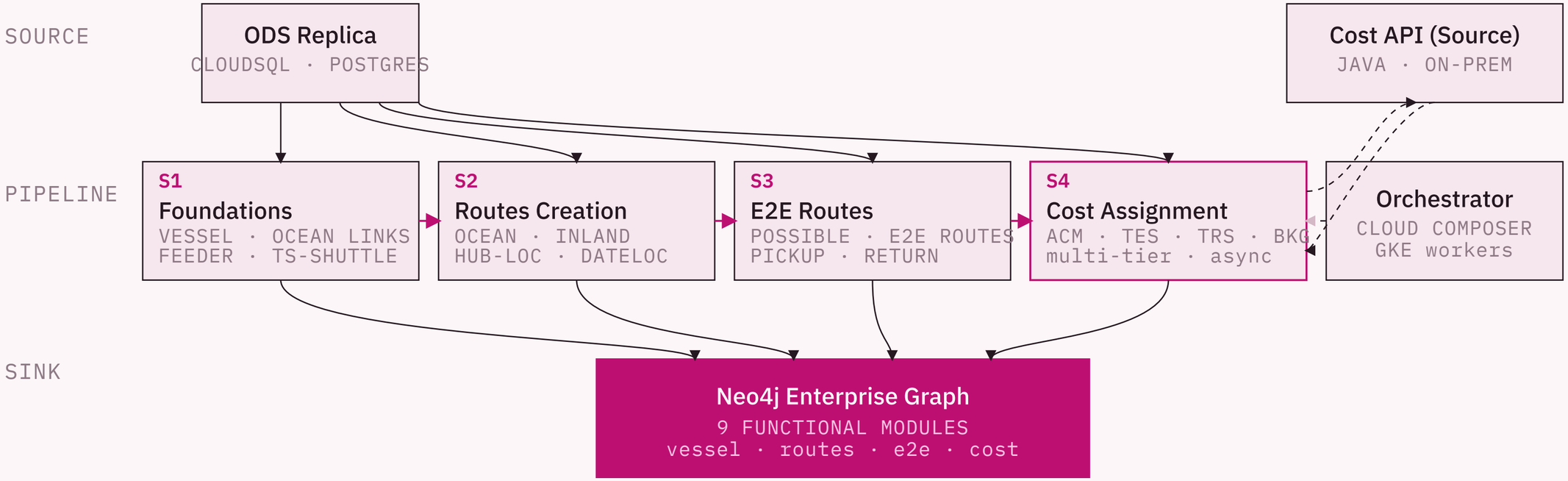
SOURCE	Oracle (on-prem)	CDC	Striim
BUS	Kafka (Confluent)	PIPELINES	Python · GKE · Cloud Run
GRAPH	Neo4j Enterprise	API · UI	NestJS · Next.js

Three complementary data pipelines — initial load, weekly route creation, near-real-time updates — feed a single graph of record.



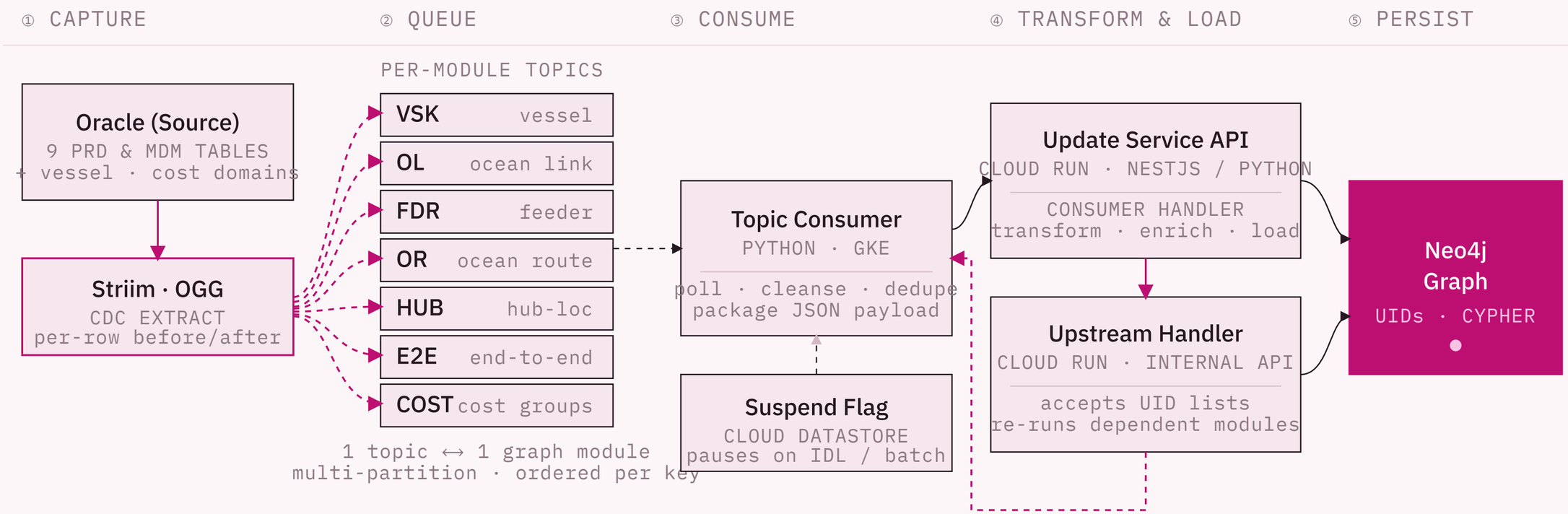
SYNC / BATCH ASYNC / QUEUE CDC PATH

A four-stream foundation load — vessel schedules, ocean & inland routes, end-to-end paths, multi-tier cost — orchestrated nightly on Cloud Composer.



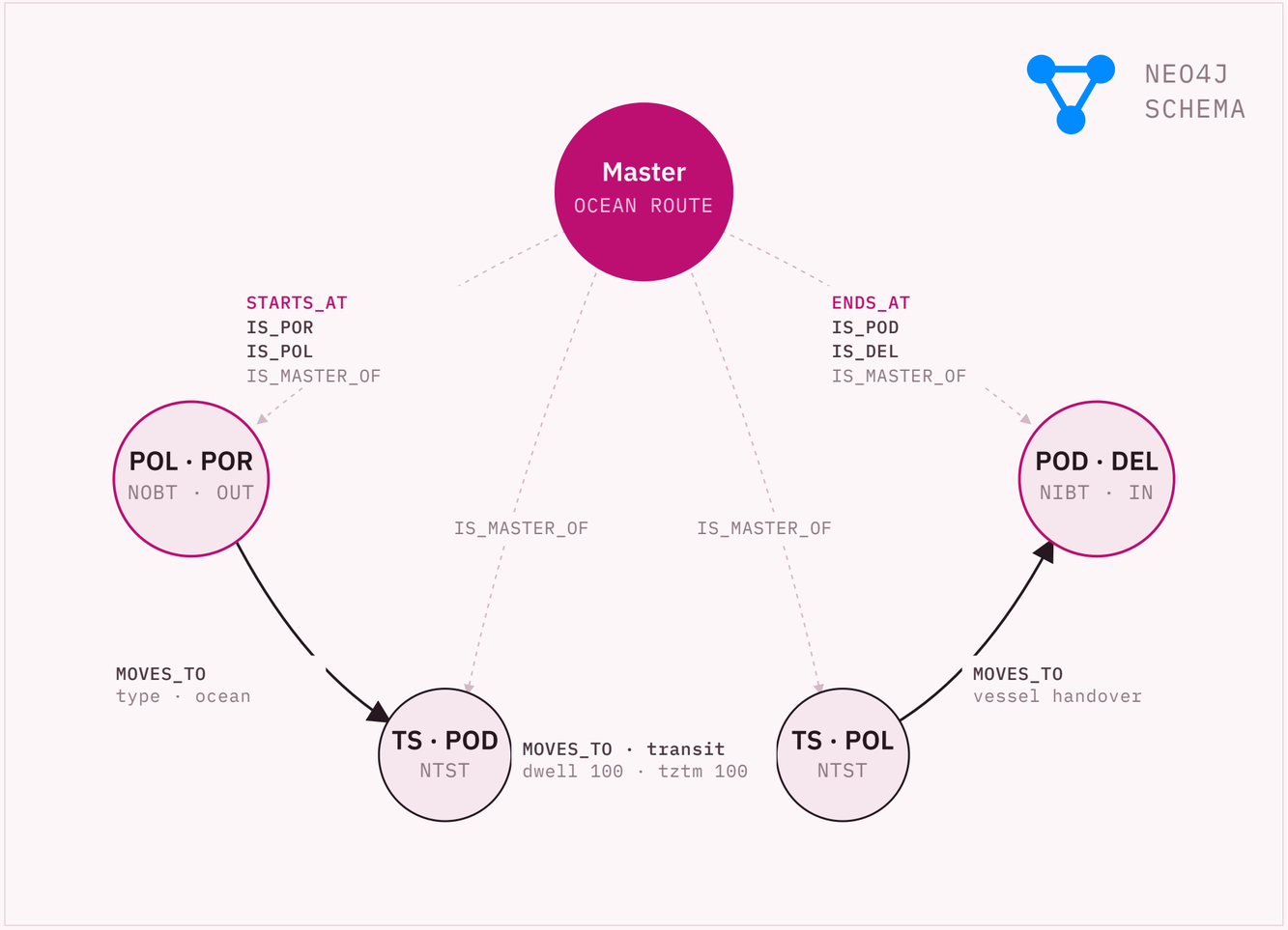
STREAM DEPENDENCY — DATA IN / OUT --- ASYNC COST CALLBACK

CDC-driven topics, minimal-work consumers, and a serverless update service that propagates change downstream through the dependency graph.



CDC PROPAGATION QUEUE / FAN-OUT SYNC API CALL

A Master route node anchors port and transhipment vertices — cost groups, movement legs and dwell times all hang off the same graph pattern.



● MASTER · OCEAN ROUTE

Top-level route entity. One per `masterCd`, with denormalised summary fields for fast filtering.

`masterCd · POR · POL · POD · DEL · sail_week`
`total_travel_time · num_TS · dgCd · tpSz`

○ PORT · POL / POR / POD / DEL

Endpoint vertices tagged with **NOBT** (outbound) or **NIBT** (inbound) cost groups; carry vessel ETB/ETD.

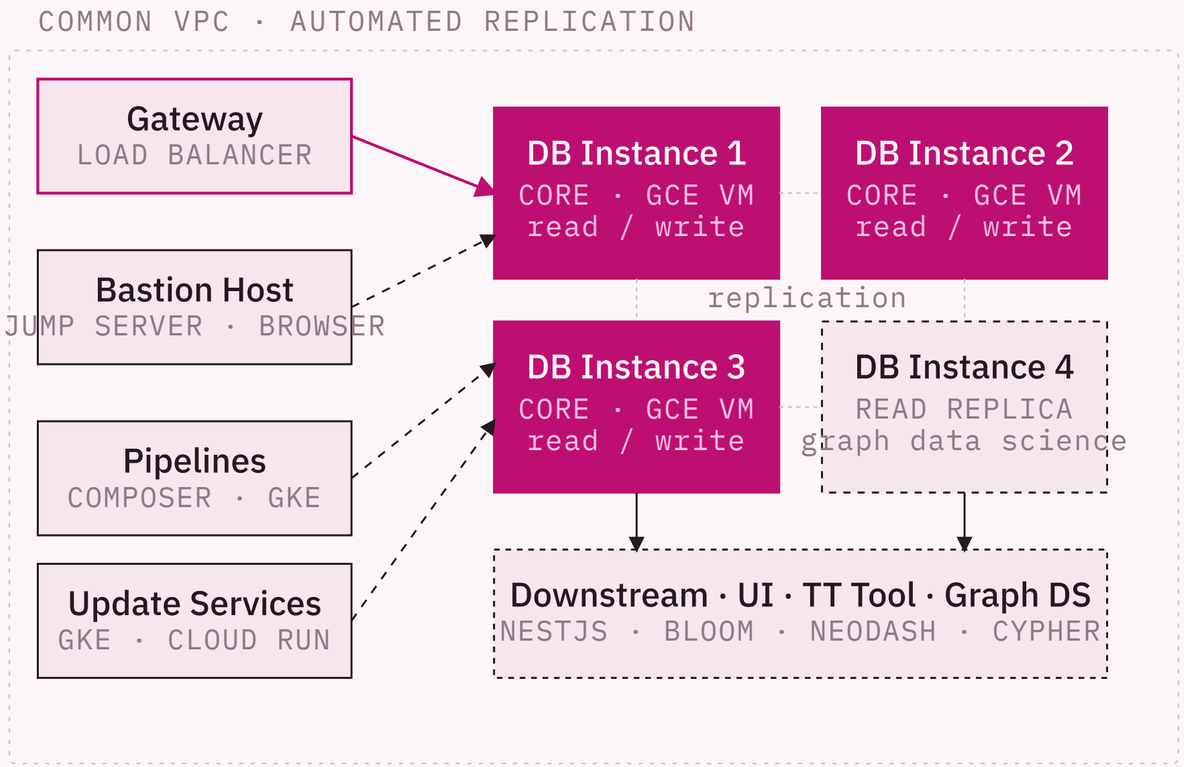
`portCd · yardCd · week · row_UID · ETB · ETD`
`iocCd · laneCd · trdCd · costActGrpCd · ioBndCd`

○ TRANSHIPMENT · NTST

Discharge / load pair at an intermediate yard. **dwell time** and **tztm** sit on the connecting `MOVES_TO` edge.

`portCd · yardCd · iocCd · laneCd · subTrdCd`
`costActGrpCd · ioBndCd · row_UID`

```
// resolve an end-to-end route by master code
MATCH (m:Master)-[:STARTS_AT]->(:POL)-[:MOVES_TO*1..]->(:POD)
WHERE m.masterCd = $code
RETURN m, nodes, relationships
```



TOPOLOGY

Three core read / write instances forming the consensus group, with a fourth read-replica reserved for Graph Data Science workloads — isolating heavy analytical reads from the transactional cluster.

ACCESS

A single load-balanced gateway fronts the cluster on a non-public VPC. Pipelines, update services and downstream consumers all enter through it; administrators reach a Neo4j Browser via a bastion jump host.

ENVIRONMENTS

Dev, Test, Stage and Prod each mirror the same topology, deployed declaratively. Aura DB was used during PoC and migrated to self-managed Enterprise once write throughput requirements were validated.

WHY A CLUSTER

Initial-load throughput, weekly batch parallelism and dynamic-update bursts have very different read/write profiles — clustering let us tune cores for write and the replica for read without compromising either path.

6× scale

ROUTES GENERATED PER WEEKLY LOAD

477k → **2.9M** end-to-end routes, on the same wall-clock budget.

−53%

WEEKLY BATCH WALL TIME

16.8 hr → **~8 hr target** after pathfinding & load optimization.

~10 min

FOUNDATION LOAD (6 MONTHS OF DATA)

Held flat while route output scaled, via GKE pod parallelism.

BENCHMARK	KEY CHANGE	ROUTES / WK	FOUNDATION LOAD	WEEKLY LOAD
Baseline	First end-to-end pass on the cluster	477,597	10.4 min	16.8 hr
Optimization pass	Re-tuned Cypher pathfinding · E2E loading rewrite · column pruning	2,900,000	12.2 min	15.4 hr
Target state	Redundant ocean route removal · 60-day max dwell · parallelized E2E	**2,900,000**	**~10 min**	**~8 hr**